



DOCUMENT DE CADRAGE

Quiz d'Æther

Le quiz de culture générale où l'intelligence artificielle crée les questions.

MODULE

**Développement d'application front /
back**

FORMATION

M2 Développement — Ynov

AUTEUR

Batiste Blactot

DATE DE RENDU

24/04/2026

DÉPÔTS GITHUB

Frontend → github.com/Batiste1998/aether-front • **Backend** →
github.com/Batiste1998/aether-api

• Sommaire

1 Brief projet

1.1 Présentation · 1.2 Arborescence · 1.3 Wireframes · 1.4 Fonctionnalités

2 Modélisation de la base de données

2.1 MCD (Merise) · 2.2 MLD · 2.3 MPD — script SQL PostgreSQL

3 Stack technique & justifications

3.1 Frontend · 3.2 Backend (Rust) · 3.3 Base de données · 3.4 Outils & infrastructure

4 Fonctionnalité IA — la génération de quiz

4.1 Description · 4.2 Choix technique · 4.3 Flux de données

En une phrase. *Quiz d'Æther* est un jeu de quiz de culture générale jouable dans le navigateur : le joueur choisit un thème — une catégorie ou un sujet libre — et une **IA (GPT-4o)** génère un **questionnaire à choix multiples** sur mesure. On répond contre le chrono, on enchaîne les bonnes réponses pour grimper au classement. Le projet couvre un **frontend React/TypeScript**, une **API Rust** et une **base PostgreSQL** relationnelle.

1 Brief projet

1.1 Présentation générale

Nom du projet

Quiz d'Æther — nom de code `aether` .

Description courte

Quiz d'Æther est une application web de quiz de culture générale. Le joueur choisit un thème (parmi des catégories proposées ou en saisissant un sujet de son choix), une difficulté et un nombre de questions ; une intelligence artificielle génère alors un questionnaire à choix multiples inédit. Chaque question est chronométrée : plus on répond vite et plus on enchaîne les bonnes réponses, plus le score grimpe. À la fin de la partie, le joueur voit son récapitulatif et se compare aux autres dans un classement. Comme les questions sont générées à la volée, le nombre de quiz possibles est virtuellement infini.

Problème résolu

Les quiz classiques (applications, jeux de société) reposent sur une banque de questions **finie** : on finit par retomber sur les mêmes, et les thèmes disponibles sont limités à ce que les concepteurs ont prévu. **Quiz d'Æther lève cette limite** : l'IA produit un quiz unique sur *n'importe quel* sujet, à la demande. La rejouabilité est totale et l'outil devient aussi un moyen ludique de réviser un domaine précis.

Public cible

- **Amateurs de quiz et de trivia** (Trivial Pursuit, Kahoot, HQ Trivia), 15–40 ans, qui cherchent des parties courtes et rejouables.
- **Joueurs occasionnels** voulant se distraire quelques minutes, sans installation, sur navigateur.
- **Étudiants & formateurs** souhaitant réviser ou animer une session sur un thème précis.

1.2 Arborescence (Arbo)

Structure de navigation de l'application. Les sections grisées nécessitent une authentification.

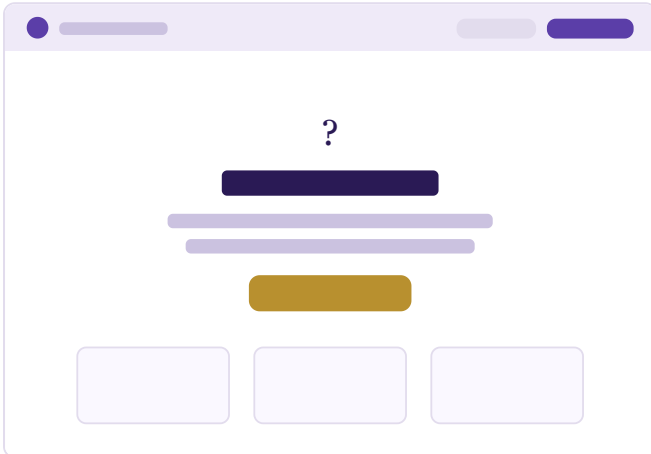
Accueil (landing publique)

```
|  présentation du concept · CTA « Jouer »
|
|— Authentification
|  |— Connexion
|  |— Inscription
|
|— Lobby / Dashboard (authentié)
|  |— Choix d'une catégorie (ou thème libre)
|  |— Réglages → difficulté · nombre de questions
|  |— Mes dernières parties (historique)
|
|— Partie (écran de quiz) (cœur de l'app)
```

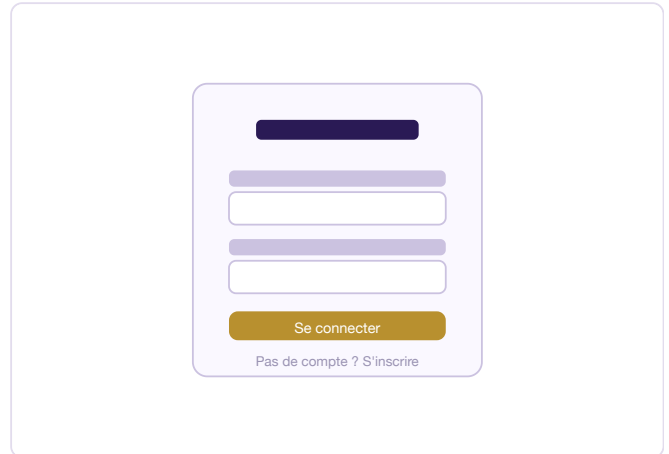
- └─ Question chronométrée → 4 propositions
- └─ Retour immédiat → bonne réponse · explication · points
- └─ Écran de résultats → score · réussite · rang
- └─ **Classement**
 - └─ Meilleurs scores, tous thèmes confondus

1.3 Wireframes

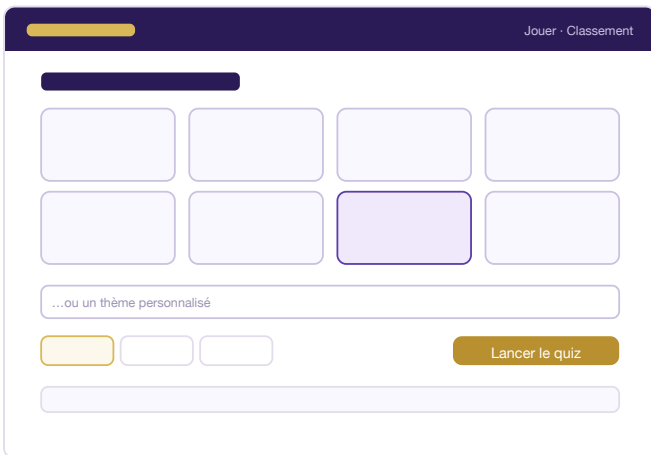
Maquettes fil-de-fer basse fidélité des cinq écrans principaux (format vectoriel, exportables vers Figma).



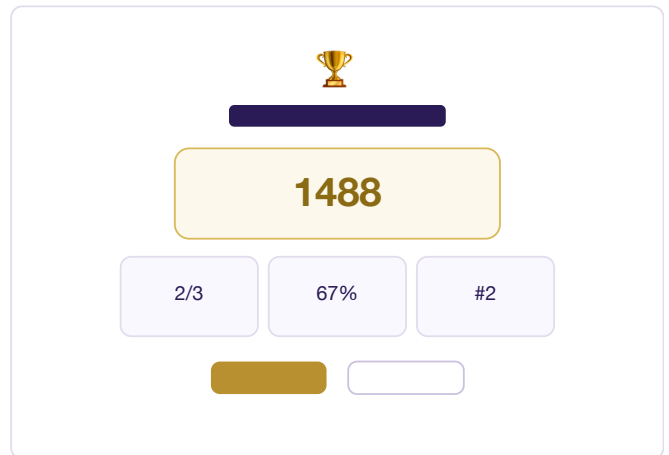
① Accueil / landing — concept, CTA « Jouer », trois arguments.



② Connexion / Inscription — formulaire centré.



③ Lobby — grille de catégories, thème libre, difficulté & lancement.



⑤ Résultats — score, réussite, rang, puis « Rejouer » / « Classement ».

Question 2/3

🔥 série x2

703 pts

Progression bar: [Progress bar showing 2/3 completed]

Four answer boxes (two highlighted green for correct answers):

- [Correct]
- [Correct]
- [Incorrect]
- [Incorrect]

✓ Bonne réponse ! +703 pts

④ Écran de quiz (vue principale) — progression & score, chrono, question, 4 propositions, retour immédiat (bonne réponse en vert).

1.4 Liste des fonctionnalités

PRIORITÉ	FONCTIONNALITÉ	DESCRIPTION	RÔLE
Must have	Inscription / Connexion	Création de compte, authentification, session sécurisée (JWT).	Tous
Must have	Choix du thème	Catégorie proposée ou thème libre, difficulté, nombre de questions.	Joueur
Must have	Génération du quiz par IA	GPT-4o produit un QCM inédit sur le thème choisi. (cf. §4)	Joueur
Must have	Jouer une partie	Questions chronométrées, choix unique, calcul du score côté serveur.	Joueur
Must have	Retour & explication	Bonne réponse révélée + explication courte après chaque question.	Joueur
Must have	Classement	Meilleurs scores, tous thèmes confondus.	Tous
Should have	Bonus de rapidité & série	Plus de points si on répond vite et qu'on enchaîne les bonnes réponses.	Joueur
Should have	Historique des parties	Liste des quiz joués et scores obtenus.	Joueur
Should have	Niveaux de difficulté	Facile / Moyen / Difficile, pris en compte dans le prompt IA.	Joueur
Nice to have	Quiz multijoueur	Salles en temps réel, plusieurs joueurs sur le même quiz (websockets).	Joueur
Nice to have	Quiz quotidien	Un quiz du jour partagé par tous, avec classement dédié.	Joueur

PRIORITÉ	FONCTIONNALITÉ	DESCRIPTION	RÔLE
Nice to have	Partage de score	Lien / image de résultat partageable sur les réseaux.	Joueur
Nice to have	Back-office d'administration	Gestion des catégories et modération des contenus.	Admin

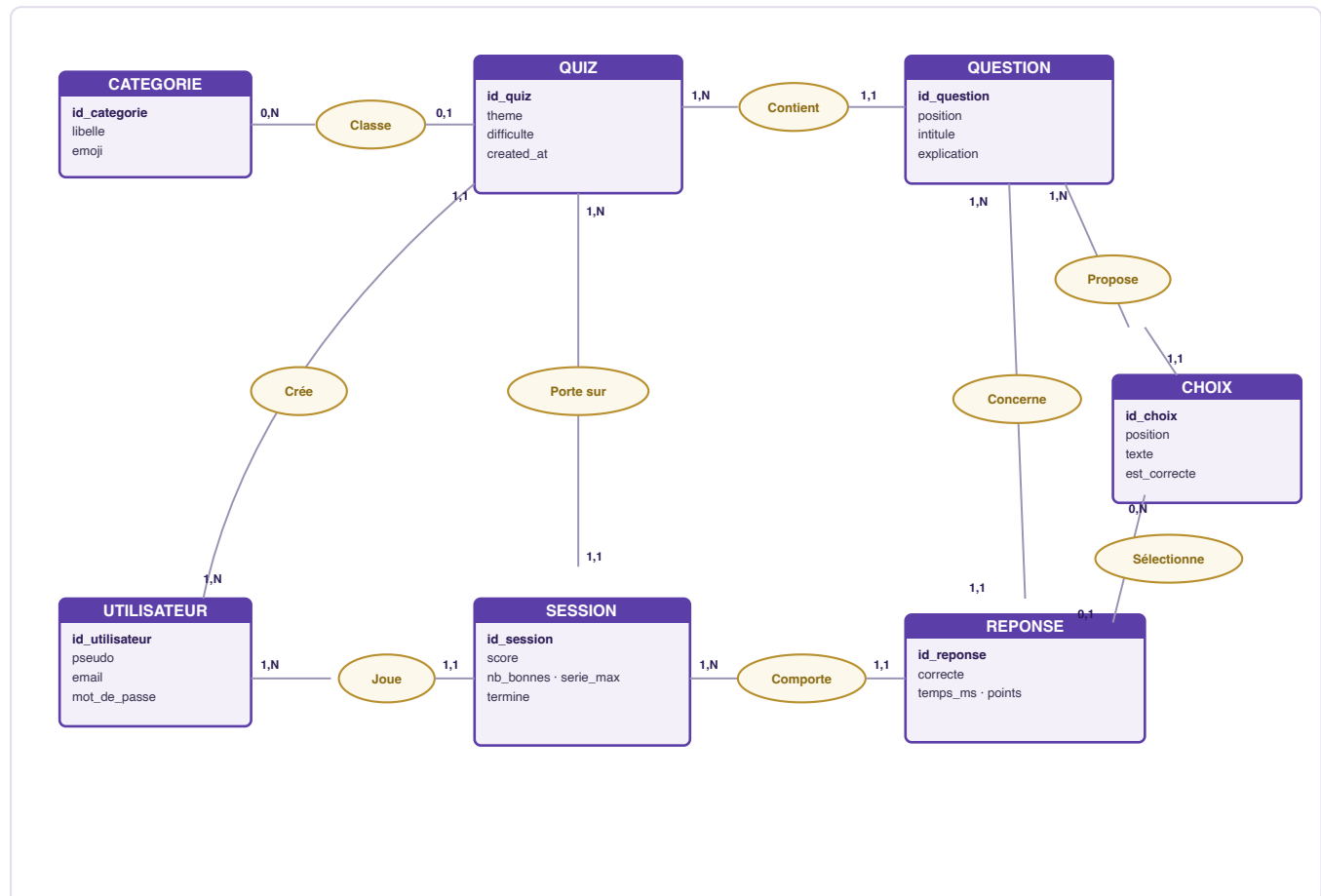
Must have indispensable au MVP ·
 Should have important, non bloquant ·
 Nice to have bonus si le temps le permet.

2 Modélisation de la base de données

Les trois niveaux de modélisation Merise : conceptuel (MCD), logique (MLD) et physique (MPD).

2.1 MCD — Modèle Conceptuel de Données

Schéma Merise : rectangles = **entités**, ovales = **associations**, chiffres = **cardinalités**. Le *quiz* est l'entité centrale : il regroupe des questions (chacune avec ses choix) et il est joué au sein de *sessions*, dont chaque *réponse* référence une question et le choix sélectionné.



MCD (Merise) — 7 entités. Sélectionne est en 0,1 côté réponse : une réponse peut être vide (temps écoulé).

Lecture des cardinalités. Un *quiz* contient 1 à N questions ; une *question* appartient à 1 seul quiz. Une *session* porte sur 1 quiz et est jouée par 1 utilisateur. Chaque *réponse* concerne 1 question et *sélectionne* 0 ou 1 choix (0 si le chrono expire avant la réponse).

2.2 MLD — Modèle Logique de Données

Traduction du MCD en tables relationnelles. Clés primaires soulignées, clés étrangères préfixées par #.

```
utilisateur (id_utilisateur, pseudo, email, mot_de_passe, role, created_at)
```

```

categorie (id_categorie, libelle, description, emoji)

quiz (id_quiz, theme, difficulte, created_at,
      #id_categorie, #id_utilisateur)

question (id_question, position, intitule, explication, #id_quiz)

choix (id_choix, position, texte, est_correcte, #id_question)

session (id_session, score, nb_bonnes, serie_courante, serie_max,
         termine, created_at, #id_quiz, #id_utilisateur)

reponse (id_reponse, correcte, temps_ms, points,
         #id_session, #id_question, #id_choix)

```

2.3 MPD — Modèle Physique de Données

Script SQL **exécutable** sur PostgreSQL 16 (c'est la migration réellement utilisée par l'application). Les énumérations sont contraintes par `CHECK`.

```

-- ===== Quiz d'Ether - schéma PostgreSQL =====

CREATE TABLE utilisateur (
    id_utilisateur SERIAL PRIMARY KEY,
    pseudo         VARCHAR(50) NOT NULL UNIQUE,
    email          VARCHAR(255) NOT NULL UNIQUE,
    mot_de_passe   VARCHAR(255) NOT NULL,    -- hash Argon2id
    role           VARCHAR(20) NOT NULL DEFAULT 'joueur'
                CHECK (role IN ('joueur','admin')),
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE categorie (
    id_categorie SERIAL PRIMARY KEY,
    libelle       VARCHAR(80) NOT NULL UNIQUE,
    description    TEXT,
    emoji         VARCHAR(8)
);

CREATE TABLE quiz (
    id_quiz SERIAL PRIMARY KEY,
    theme     VARCHAR(120) NOT NULL,
    difficulte VARCHAR(20) NOT NULL
                CHECK (difficulte IN ('facile','moyen','difficile')),
    id_categorie INT REFERENCES categorie(id_categorie),
    id_utilisateur INT NOT NULL REFERENCES utilisateur(id_utilisateur) ON DELETE CASCADE,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE question (
    id_question SERIAL PRIMARY KEY,
    id_quiz     INT NOT NULL REFERENCES quiz(id_quiz) ON DELETE CASCADE,

```



```

    position      INT NOT NULL,
    intitule      TEXT NOT NULL,
    explication   TEXT,
    UNIQUE (id_quiz, position)
);

CREATE TABLE choix (
    id_choix      SERIAL PRIMARY KEY,
    id_question   INT NOT NULL REFERENCES question(id_question) ON DELETE CASCADE,
    position      INT NOT NULL,
    texte         TEXT NOT NULL,
    est_correcte  BOOLEAN NOT NULL DEFAULT false,
    UNIQUE (id_question, position)
);

CREATE TABLE session (
    id_session    SERIAL PRIMARY KEY,
    id_quiz       INT NOT NULL REFERENCES quiz(id_quiz) ON DELETE CASCADE,
    id_utilisateur INT NOT NULL REFERENCES utilisateur(id_utilisateur) ON DELETE CASCADE,
    score         INT NOT NULL DEFAULT 0,
    nb_bonnes     INT NOT NULL DEFAULT 0,
    serie_courante INT NOT NULL DEFAULT 0,
    serie_max     INT NOT NULL DEFAULT 0,
    termine       BOOLEAN NOT NULL DEFAULT false,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE reponse (
    id_reponse    SERIAL PRIMARY KEY,
    id_session    INT NOT NULL REFERENCES session(id_session) ON DELETE CASCADE,
    id_question   INT NOT NULL REFERENCES question(id_question),
    id_choix      INT REFERENCES choix(id_choix),      -- NULL si temps écoulé
    correcte      BOOLEAN NOT NULL,
    temps_ms      INT NOT NULL,
    points        INT NOT NULL,
    UNIQUE (id_session, id_question)      -- une seule réponse par question
);

-- ===== Index utiles =====
CREATE INDEX idx_question_quiz ON question(id_quiz);
CREATE INDEX idx_choix_question ON choix(id_question);
CREATE INDEX idx_session_score ON session(score DESC);
CREATE INDEX idx_reponse_session ON reponse(id_session);

```

Note : la contrainte `UNIQUE (id_session, id_question)` empêche de répondre deux fois à la même question (anti-triche basique).

3 Stack technique & justifications

Architecture découplée : SPA React (client) $\rightleftharpoons$ API REST Rust $\rightleftharpoons$ PostgreSQL, avec appels sortants vers l'API OpenAI.

Le pari Rust. Le backend est écrit en **Rust**. Ce n'est pas le choix « par défaut » d'un projet web étudiant — c'est un choix assumé : sûreté mémoire sans ramasse-miettes, vérification forte par le compilateur, et un coût d'exécution minimale. C'est précieux pour un serveur qui valide les réponses, calcule les scores et orchestre des appels réseau coûteux vers GPT-4o.

3.1 Frontend

ÉLÉMENT	CHOIX	JUSTIFICATION
Bibliothèque UI	React 19	Rendu par composants idéal pour une interface qui change d'état à chaque question (chrono, sélection, retour) ; écosystème mature et employabilité.
Langage	TypeScript	Typage statique : les structures échangées avec l'API Rust (typée elle aussi) sont décrites côté client, éliminant une classe entière de bugs d'intégration.
Build / dev	Vite	Démarrage instantané, HMR rapide, build optimisé — bien plus léger que Webpack/CRA.
UI / CSS	Tailwind CSS	Prototypage rapide et cohérent avec les wireframes ; thème sombre décliné via les design tokens.
Routage	React Router	Standard de fait pour le routage SPA (accueil, lobby, quiz, classement) avec routes protégées par authentification.
État serveur	TanStack Query	Cache, invalidation et états de chargement gérés proprement — essentiel pour l'attente de la génération IA (latence) et la synchro classement/historique.
État client	Zustand	Store minimaliste pour la session d'authentification, sans la verbosité de Redux.

3.2 Backend

ÉLÉMENT	CHOIX	JUSTIFICATION
Langage	Rust (édition 2021)	Sûreté mémoire et concurrence garanties par le compilateur ; ces erreurs sont éliminées <i>avant</i> l'exécution sur un serveur multi-joueurs.

ÉLÉMENT	CHOIX	JUSTIFICATION
Framework web	Axum	Framework asynchrone (Tokio) ergonomique et performant ; le modèle async convient à un backend dont la latence dominante est l'attente d'OpenAI (I/O-bound).
Accès BDD	SQLx	SQL async, sans ORM lourd : on garde le contrôle du SQL, cohérent avec le travail de modélisation du §2.
Authentification	JWT (<code>jsonwebtoken</code>)	Authentification stateless adaptée à une API REST consommée par une SPA ; le token porte l'identité et le rôle, vérifiés par un middleware Axum.
Hachage mots de passe	Argon2id (<code>argon2</code>)	Fonction de hachage lauréate du Password Hashing Competition, recommandée par l'OWASP.
Client IA	<code>async-openai</code>	Client Rust typé pour l'API OpenAI (chat completions, sortie JSON structurée) — intégration directe de la génération de quiz (cf. §4).
Sérialisation	Serde	(Dé)sérialisation JSON de référence en Rust ; mappe les corps de requête/réponse et la sortie de l'IA.

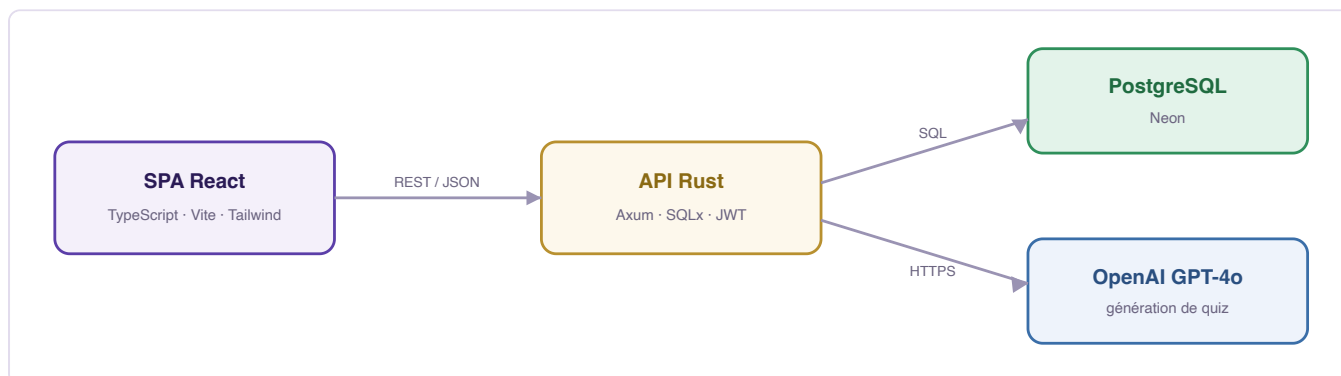
Honnêteté sur l'arbitrage. Rust a une courbe d'apprentissage plus raide que Node.js et un temps de compilation plus long. Ce coût est assumé : le projet est aussi une montée en compétence sur un langage système recherché, et la rigueur du compilateur réduit drastiquement le débogage en aval.

3.3 Base de données

ÉLÉMENT	CHOIX	JUSTIFICATION
SGBD	PostgreSQL 16	Relationnel robuste, parfaitement adapté au schéma normalisé du §2 (intégrité référentielle, contraintes <code>CHECK</code> et <code>UNIQUE</code> pour l'anti-triche).
Hébergement	Neon (dev & prod)	PostgreSQL serverless avec branches de base et offre gratuite généreuse — idéal en contexte étudiant.
Migrations	<code>sqlx-cli</code>	Migrations versionnées dans le dépôt, rejouées automatiquement au démarrage de l'API.

3.4 Outils & infrastructure

ÉLÉMENT	CHOIX	JUSTIFICATION
Versioning	Git + GitHub (2 dépôts)	Séparation front / back, historique, revue par Pull Request.
Intégration continue	GitHub Actions	Back : <code>fmt</code> + <code>clippy</code> + tests + build. Front : typecheck + tests + build. À chaque push.
Conteneurisation	Docker	PostgreSQL en local via <code>docker compose</code> ; image du binaire Rust reproductible.
Déploiement back	Fly.io / Shuttle.rs	Déploiement simple d'un binaire Rust conteneurisé.
Déploiement front	Vercel	Déploiement de la SPA optimisé, previews par PR, CDN global.
Gestion de projet	GitHub Projects	Backlog priorisé (Must/Should/Nice) lié aux issues et PR.



Architecture générale — SPA $\rightleftharpoons$ API Rust $\rightleftharpoons$ PostgreSQL, avec appels sortants vers OpenAI.

4 Fonctionnalité IA — la génération de quiz

L'IA n'est pas un accessoire : c'est elle qui crée le contenu du jeu. Sans elle, il n'y a pas de quiz.

4.1 Description de la fonctionnalité

À partir d'un **thème**, d'une **difficulté** et d'un **nombre de questions**, une IA (GPT-4o) génère un **questionnaire à choix multiples** complet : pour chaque question, un intitulé, quatre propositions (une seule correcte) et une courte explication. Le quiz est ensuite stocké en base et joué par l'utilisateur.

- **Problème résolu** : des questions inédites sur n'importe quel sujet, à l'infini — fini les banques de questions figées et répétitives.
- **Moment d'intervention** : au lancement d'une partie depuis le lobby (wireframe ③). C'est l'étape qui crée le contenu joué juste après.

4.2 Choix technique

MODÈLE	OpenAI GPT-4o — solide en culture générale et en production de sorties structurées fiables. Repli possible sur <code>gpt-4o-mini</code> pour réduire le coût.
OÙ ?	Côté backend (Rust) . Le client React ne parle <i>jamais</i> directement à OpenAI : la clé API reste secrète sur le serveur, qui contrôle le prompt, valide la sortie et — surtout — ne renvoie jamais la bonne réponse au client avant qu'il ait répondu (anti-triche).
COMMENT ?	Le crate <code>async-openai</code> appelle l'endpoint <i>Chat Completions</i> avec <code>response_format = json_object</code> . La sortie est validée (4 propositions par question, index de bonne réponse cohérent) puis enregistrée dans les tables <code>quiz</code> , <code>question</code> et <code>choix</code> .

4.3 Flux de données — entrée & sortie

Entrée envoyée au modèle

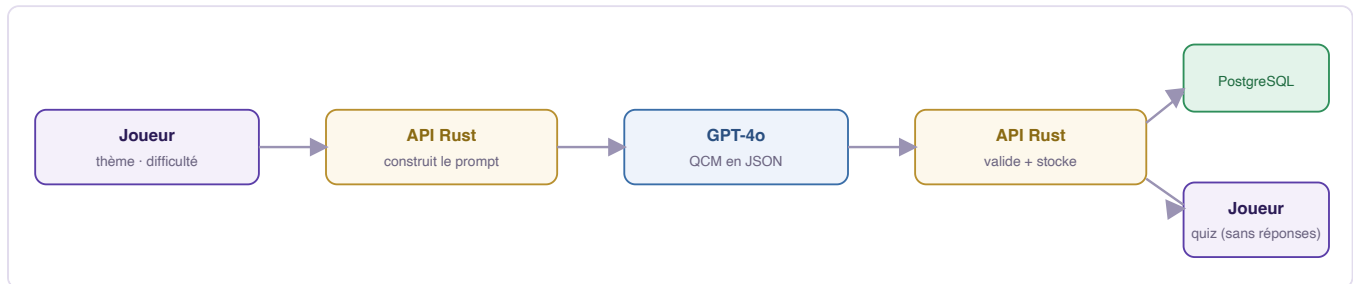
- **Prompt système** : rôle, règles (4 choix, une seule bonne réponse, explication, pas d'ambiguïté), format de sortie imposé.
- **Thème** : catégorie choisie ou sujet libre saisi par le joueur.
- **Difficulté** : facile / moyen / difficile.
- **Nombre de questions** : borné entre 3 et 10.

Sortie récupérée (JSON structuré)

- **questions** : liste d'objets question.
- **intitule** : l'énoncé de la question.
- **choix** : tableau de 4 propositions.
- **bonne_reponse** : index (0-3) de la bonne proposition.
- **explication** : justification courte de la réponse.

Le backend valide puis répartit cette sortie dans le schéma relationnel ; côté `choix`, l'attribut `est_correcte` est positionné selon l'index renvoyé. Lors du jeu, le calcul du score (base + bonus de rapidité + bonus de série) est effectué **côté serveur** à chaque réponse.

```
// Sortie structurée imposée au modèle (response_format json_object) – extrait
{
  "questions": [
    {
      "intitule": "Quel astronome a démontré que la Terre tourne autour du Soleil ?",
      "choix": ["Galilée", "Nicolas Copernic", "Isaac Newton", "Ptolémée"],
      "bonne_reponse": 1,
      "explication": "Copernic a proposé le modèle héliocentrique au XVIe siècle."
    }
  ]
}
```



Génération d'un quiz — l'API Rust construit le prompt, appelle GPT-4o, valide et stocke, puis renvoie le quiz au joueur sans les bonnes réponses.

Implémentation réelle. Cette fonctionnalité est effectivement développée : c'est le moteur de contenu du jeu, sans lequel l'application n'a pas de raison d'être. Les fonctionnalités IA bonus (quiz quotidien partagé, génération d'illustration de thème) viendront l'enrichir si le temps le permet.